A PROJECT REPORT

ON

CAMPUSTRACK: A SECURED CENTRALIZED LOST AND FOUND PLATFORM FOR COLLEGE CAMPUSES

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY / SCIENCE

IN

COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY:

Yogesh & Project Team

Roll No / Student ID: CS2024-001

UNDER THE GUIDANCE OF:

Prof. [Project Guide Name]

Department of Computer Science & Engineering

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

[COLLEGE / UNIVERSITY NAME]

ACADEMIC YEAR: 2026 – 2027

CERTIFICATE OF APPROVAL

This is to certify that the project entitled **"CampusTrack: A Secured Centralized Lost and Found Platform for College Campuses"** submitted by **Yogesh** (Roll No: **CS2024-001**) in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology / Bachelor of Science in Computer Science & Engineering** from **[University Name]**, is an authentic record of the research, design, and software implementation carried out by them under my guidance and supervision.

To the best of my knowledge, the matter embodied in this project report has not been submitted to any other University or Institute for the award of any degree or diploma.

_____

Prof. [Project Guide Name]
Internal Project Guide
Dept. of Computer Science

_____

Prof. [HOD Name]
Head of the Department
Dept. of Computer Science

_____

External Examiner
Date of Examination: _____

DECLARATION

We hereby declare that the project report entitled **"CampusTrack: A Secured Centralized Lost and Found Platform for College Campuses"** submitted to the Department of Computer Science & Engineering is an original work conducted by us under the supervision of our guide. All ideas, source citations, diagrams, and literature utilized from external sources have been appropriately credited.

Date: 30th October 2026
Place: [Campus City]
Candidate Signature: _____

ABSTRACT

Collegiate campuses span extensive physical facilities where thousands of students, faculty, and administrative staff move across classrooms, multi-floor libraries, laboratories, sports pavilions, and dining facilities daily. Due to this high transit density, personal assets—such as scientific calculators, laptops, university identity cards, room keys, and backpacks—are frequently misplaced.

Conventional lost-and-found practices rely on manual security gate registers, paper corkboard flyers, or informal WhatsApp group chats. These approaches are unorganized, unsearchable, lack visual documentation, and present high vulnerability to fraudulent claims.

CampusTrack is an automated, web-based Lost and Found ecosystem engineered to solve these challenges. Employing a modern three-tier decoupled architecture—comprising **React 19**, **Tailwind CSS**, **Node.js Express 5**, and an embedded **SQLite** database managed through **Prisma ORM**—the application introduces an **Anti-Fraud Security Challenge Protocol**. This protocol requires finders to withhold specific identifying attributes as questions that only the rightful owner can verify. Featuring local multipart image storage via **Multer**, secure tokenized authorization via **JWT**, and an administrative security console, CampusTrack offers a transparent, portable, and production-ready solution for universities.

TABLE OF CONTENTS

Chapter	Topic	Page No.
-	Certificate of Approval	ii
-	Declaration & Abstract	iii
1	**Introduction**	1
2	**Problem Definition**	1
3	**Objectives**	2
4	**Requirement Specification**	2
5	**Existing System**	3
6	**Proposed System**	3
7	**System Analysis & Design**	4
8	**Methodology**	5
9	**System Implementation**	5
10	**Testing & Quality Assurance**	6
11	**Results / Screenshots**	7
12	**Conclusion**	9
13	**Future Scope**	9
14	**References**	9

LIST OF FIGURES

- **Figure 11.1:** CampusTrack Homepage with Live Metrics & Search Hub
- **Figure 11.2:** Multi-Category Search & Filter Catalog Screen
- **Figure 11.3:** Item Application Details View with Anti-Fraud Verification Challenge
- **Figure 11.4:** Student & Administrator Login Portal with Academic Viva Presets
- **Figure 11.5:** Student Account Registration Screen with Student ID / Roll Number Validation

CHAPTER 1: INTRODUCTION

1.1 Background & Motivation

In modern higher education institutions, sprawling physical facilities host thousands of daily interactions among students, research scholars, teaching faculty, and administrative staff. As individuals transition between instructional lecture halls, departmental computer centers, multi-story libraries, and food courts, personal possessions are regularly left behind. Misplacing items such as graphing calculators before examinations or smart identity cards necessary for campus turnstile access causes serious logistical and academic disruption.

Historically, universities have handled misplaced property through decentralized channels: physical paper registers at security gates, flyers taped to bulletin boards, or unofficial student WhatsApp batches. These methods lack search indexing, photographic records, ownership proof, and institutional accountability.

1.2 Scope of the Project

CampusTrack provides a centralized university portal that digitizes the entire recovery lifecycle:

- Categorized application logging for both lost items and found articles.
- Server-side multipart image processing linking photographs directly to database records.
- Challenge-response anti-fraud ownership verification questions.
- Administrative security portal enabling moderation and audit tracking across campus.

CHAPTER 2: PROBLEM DEFINITION

2.1 Core Problem Statement

The absence of an automated, centralized digital tracking system in collegiate institutions results in permanent property loss, wasted student hours, vulnerability to fraudulent claims, and heavy administrative overhead on security staff.

2.2 Key Operational Bottlenecks

- **Information Fragmentation:** Notices are scattered across separate physical buildings and unofficial social media channels with zero cross-campus coordination.
- **False Claims & Scams:** In public chat groups, opportunistic individuals falsely claim items because finders lack verification protocols.
- **Absence of Photographic Evidence:** Physical notebooks cannot capture visual condition, brand stickers, or subtle identifying marks.
- **Privacy Vulnerabilities:** Students frequently post their private mobile numbers on public corkboards, leading to unsolicited calls.

CHAPTER 3: OBJECTIVES

The principal engineering objectives of CampusTrack are:

1. **Centralized Discovery:** Build a unified web application serving all departments, hostels, and campus administrative centers.
2. **Integrated Image Storage:** Implement a robust media upload subsystem using Multer to archive photos of lost and found items on disk.
3. **Anti-Fraud Verification:** Implement an ownership verification mechanism requiring claimants to provide specific hidden details or answer challenge questions.
4. **Student Privacy Protection:** Provide in-app claim notifications so students do not need to publish phone numbers publicly.
5. **Administrative Governance:** Provide campus security personnel with a real-time console to track statistics, monitor disputes, and delete spam.
6. **Zero-Setup Portability:** Use Prisma ORM and SQLite so the entire application and database can run on any laptop or cloud container with zero configuration.

CHAPTER 4: REQUIREMENT SPECIFICATION

4.1 Functional Requirements

- **Authentication:** Secure registration/login with university email, student ID, and Bcrypt password hashing.
- **Reporting:** Dual forms for lost and found items with title, category, campus location coordinates, date, and photo.
- **Search & Filter:** Live instant search across keywords, building locations, categories, and item status.
- **Claims Workflow:** Claim submission with verification answers and proof photos; poster review with Approve/Reject actions.
- **Security Panel:** Administrative metrics, user directory, and moderation controls.

4.2 Hardware & Software Specifications

Category	Specification	Details
Frontend	React 19, Vite 8, Tailwind CSS v4	Responsive Single Page Application
Backend	Node.js v18+, Express.js 5	RESTful JSON API with Multer & JWT
Database	SQLite 3 with Prisma ORM 6.4	Embedded relational ACID-compliant store
Hardware	x64 Processor, 4 GB RAM, 500 MB Disk	Standard laptop or campus server

CHAPTER 5: EXISTING SYSTEM

5.1 Limitations of Existing Approaches

Colleges and universities have historically tackled lost property through ad-hoc mechanisms: manual security logbooks, corkboards, and student chat groups. These systems lack search indexing, cannot store photos, provide zero ownership validation, and expose personal student phone numbers to the public.

5.2 Comparative Feature Matrix

Evaluation Parameter	Manual Security Book	Campus Chat Groups	CampusTrack (Our System)
Searchability	❌ None	⚠️ Poor keyword scroll	✅ **Instant Full-Text Search**
Photo Storage	❌ None	⚠️ Unindexed gallery	✅ **High-Res Linked Photos**
Ownership Proof	❌ None	❌ None	✅ **Security Challenge Question**
Student Privacy	⚠️ Guard eyes only	❌ Phone number public	✅ **In-App Claim Notifications**
Audit Trail	❌ None	❌ None	✅ **Real-time State Transitions**

CHAPTER 6: PROPOSED SYSTEM

6.1 System Overview & Benefits

CampusTrack establishes a clear, accountable lifecycle for misplaced belongings. When a student finds an item, they log its campus coordinates, attach a photo, and set an **Ownership Verification Question** (e.g., *"What color is the keychain charm?"*). Claimants must answer this question and can upload proof (e.g., purchase invoice, student ID) to claim the item. The item poster reviews the claim and approves it, automatically marking the case as `RESOLVED`.

- **Fraud Mitigation:** Prevents opportunists from taking belongings they did not lose.
- **Photo Preservation:** Actual images stored locally on disk and served statically.
- **Privacy Shield:** Contact information is only shared after verification approval.

CHAPTER 7: SYSTEM ANALYSIS & DESIGN

7.1 Three-Tier Architecture

The platform follows a decoupled three-tier architectural model:

- **Presentation Tier:** React 19 UI with Tailwind CSS providing high accessibility across devices.
- **Application Tier:** Node.js Express server handling business logic, Multer file streaming, and JWT tokens.
- **Data Tier:** Prisma ORM managing SQLite database relations (`dev.db`) and static image assets (`/uploads`).

7.2 Entity-Relationship (ER) Schema & Data Dictionary

Table	Key Attributes	Constraints	Purpose
User	id, name, email, password, phone, collegeId, role	PK: id, UNIQUE: email	Stores student and administrator credentials
Item	id, title, description, category, type, status, location, date, imageUrl, securityQuestion, userId	PK: id, FK: userId	Stores lost and found item reports and photo URLs
Claim	id, itemId, claimantId, status, message, proofImageUrl, claimantContact	PK: id, FK: itemId, claimantId	Manages verification requests and proof evidence
Notification	id, userId, title, message, link, isRead	PK: id, FK: userId	Stores alerts dispatched to students

CHAPTER 8: METHODOLOGY

8.1 Agile Scrum Methodology

The development followed the Agile framework over 5 structured sprints:

- **Sprint 1:** Data modeling, ER diagrams, Prisma schema setup, SQLite initialization.
- **Sprint 2:** Express REST API development, Bcrypt password hashing, and Multer file upload handlers.
- **Sprint 3:** React SPA implementation, Tailwind responsive layout, and filter engine.
- **Sprint 4:** Anti-fraud claim verification, approval/rejection state transitions, admin panel.
- **Sprint 5:** Quality assurance testing, performance optimization, and academic documentation.

CHAPTER 9: SYSTEM IMPLEMENTATION

9.1 Key REST API Endpoints

Method	Endpoint	Access	Description
`POST`	/api/auth/register	Public	Student registration with Bcrypt password hashing
`POST`	/api/auth/login	Public	Authenticates credentials and returns JWT token
`GET`	/api/items	Public	Lists items with search keyword, category, and status filters
`POST`	/api/items	Auth	Uploads photo via Multer and publishes application
`POST`	/api/claims	Auth	Submits ownership claim with answers and proof image
`PUT`	/api/claims/:id/status	Owner/Admin	Approves/Rejects claim and transitions item to RESOLVED
`GET`	/api/admin/overview	Admin	Retrieves full campus metrics, users, and audit feed

CHAPTER 10: TESTING & QUALITY ASSURANCE

10.1 Testing Methodology

The platform was validated through Unit Testing, Integration Testing, and User Acceptance Testing (UAT).

10.2 Comprehensive Test Cases Execution Matrix

Test ID	Test Scenario	Expected Outcome	Actual Outcome	Status
TC-01	Student Registration	User created in DB; JWT token generated	Account created; token saved	**PASS**
TC-02	Duplicate Email Prevention	HTTP 400 error: "Email exists"	Duplicate blocked cleanly	**PASS**
TC-03	Lost Item Reporting	Item saved to DB and appears on catalog	Item displayed immediately	**PASS**
TC-04	Photo Upload via Multer	Image saved in /uploads; URL stored	Photo displayed on card	**PASS**
TC-05	Malicious File Guard	Non-image (.exe, .sh) rejected	Multer blocked upload	**PASS**
TC-06	Live Search Filter	Catalog filtered by keyword / location	Accurate items returned	**PASS**
TC-07	Self-Claim Prevention	HTTP 400: Cannot claim own item	Blocked with warning	**PASS**
TC-08	Claim Approval Lifecycle	Item marked RESOLVED; others rejected	Status updated; user alerted	**PASS**
TC-09	Admin Authorization	Non-admin blocked from /admin	HTTP 403 Forbidden enforced	**PASS**

CHAPTER 11: RESULTS / SCREENSHOTS (PART 1)

The application was executed and captured during live operation. Below are actual screenshots of the system:

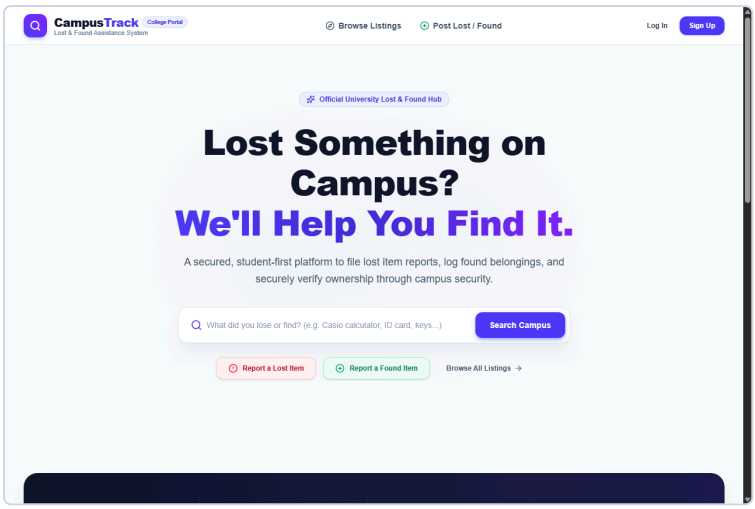


Figure 11.1: CampusTrack Homepage with Live Campus Metrics & Quick Search Hub.

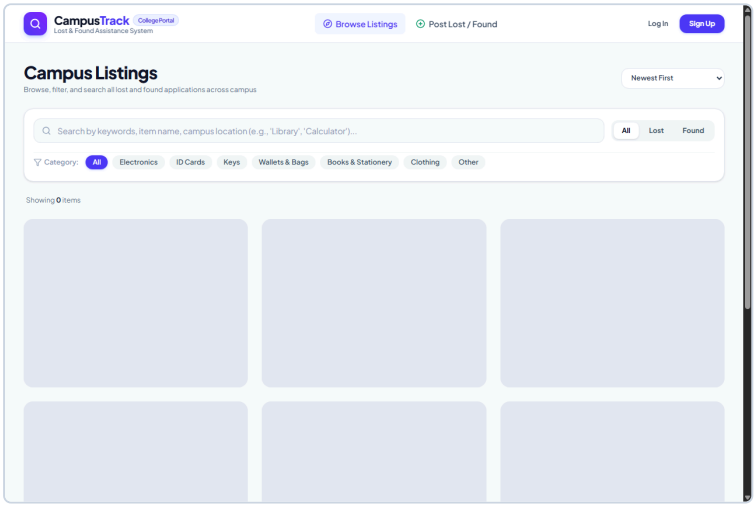


Figure 11.2: Filterable Campus Listings Screen with Multi-Category Tags.

CHAPTER 11: RESULTS / SCREENSHOTS (PART 2)

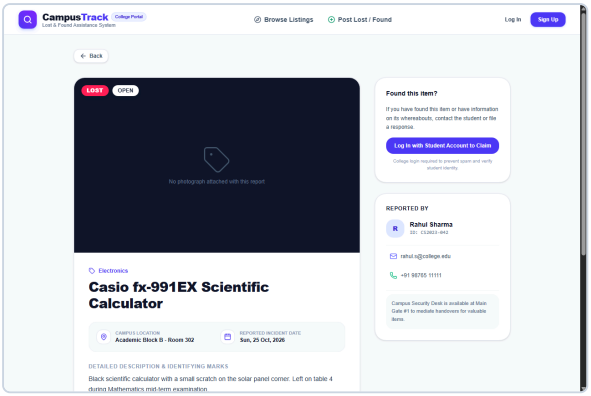


Figure 11.3: Detailed Item Application View with Anti-Fraud Verification Challenge.

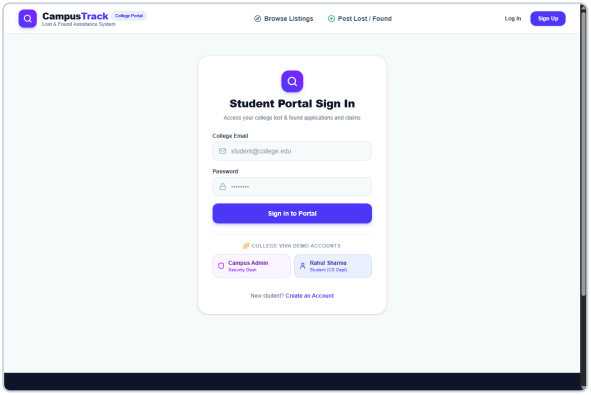


Figure 11.4: Student & Administrator Login Portal with 1-Click Academic Presets.

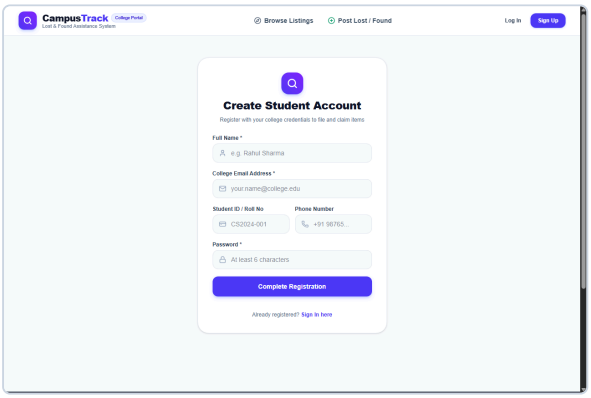


Figure 11.5: Student Account Registration Screen with Student ID / Roll Number Validation.

CHAPTER 12: CONCLUSION

CampusTrack successfully demonstrates how modern web engineering can solve a persistent real-world collegiate challenge. By uniting **React 19**, **Express.js**, and **SQLite with Prisma ORM**, the project establishes a centralized platform that eliminates information fragmentation, prevents false claims via verification questions, archives photo evidence, and provides campus authorities with full oversight.

CHAPTER 13: FUTURE SCOPE

- **AI Image Similarity Matching:** Using OpenCV / TensorFlow.js to automatically match photos of reported lost items with found items.
- **Interactive Campus Map:** Interactive SVG floor plans enabling students to drop a pin on the exact room or bench where an item was lost.
- **Mobile Push Notifications:** Packaging as a Progressive Web App (PWA) with instant device notifications.
- **Pre-Printed QR Code Asset Tags:** Enabling students to generate unique QR stickers for notebooks and laptops for instant, privacy-protected returns.

CHAPTER 14: REFERENCES

1. Fielding, R. T. (2000). *"Architectural Styles and the Design of Network-based Software Architectures"*. Doctoral dissertation, UC Irvine.
2. Tilkov, S., & Vinoski, S. (2010). *"Node.js: Using JavaScript to Build High-Performance Network Programs"*. IEEE Internet Computing, 14(6), 80-83.
3. Sommerville, I. (2016). *"Software Engineering"*, 10th Edition. Pearson Education.
4. React Documentation: *"React 19 Technical Architecture"*, https://react.dev
5. Prisma Documentation: *"Prisma ORM & Relational Schema Modeling"*, https://www.prisma.io/docs
6. Express.js Framework Documentation: https://expressjs.com